Intermediate Stations

in Flatland



course:
`Railway Scheduling`
group name:
`Ready To Depart`

by
Emil Zank
emil.zank@uni-potsdam.de

March 2026

Abstract

For the course *Railway Scheduling* the participants are working with the framework *Flatland* in combination with *Answer Set Programming* (ASP) to route trains. Whilst attending the course my main focus became adding *intermediate stations* to Flatland. Prior, trains only had a given spawn and goal within a Flatland environment. There were no other stations or way points, that a train had to visit. This report will describe my work throughout the semester as member of the single person group *ready to depart*. A general concept on how to add intermediate stations is described and some Flatland maps with implemented intermediate stations are shown. Yet my encoding to solve these environments is rather slow, so that further work regarding this issue remains necessary.

1 Introduction

Flatland is an open source toolkit to simulate train movements on some given two-dimensional map. Originally, Flatland started as a competition to schedule multiple trains from different start positions to different goal positions. The goal is to come up with an efficient solution while avoiding collisions between different trains. In general Flatland is an abstract modeling of the reality, thus the whole complexity of real train scheduling is not fully covered. For instance the given environments do not force the trains to take any specific routes or even to stop at any kind of intermediate stations. Further the tracks are placed on a grid based map, that is not capable of modeling train length or tunnels. However in newer versions of Flatland certain new elements as train speed or malfunctions can be modeled. [1] [2]

Throughout the *Railway Scheduling* course during the winter semester 2025/26 I tried to add intermediate stations to Flatland. Usually trains get only assigned some sort of start and end *stations*; my goal was to force trains to stop at additional stations on the way as well. Thus allowing the creator of a Flatland instance to list some arbitrary intermediate station at which trains have to stop. The instances we used for the course were represented using an *Answer Set Programming* (ASP) encoding.

At first my idea was to implement *terminus stations* in Flatland, in the following simply referred to as *terminus*. And a terminus being a station, which a train can only leave through reversing out. In particular I was thinking of modeling some larger European termini like *Frankfurt (Main) Hauptbahnhof* or *Paris Gare du Nord*. But for the sufficient modeling of reality of termini some prerequisites were missing in my opinion:

- change train direction of travel
- train length
- platform/ track length
- train speed
- tunnel/ bridge track type

Currently there is no feature to turn a train around in Flatland; neither any kind of logical nor animated support. Also for modeling termini it is important to take into account, that trains with a lot of carriages and therefore

a fair length can not stop at all platforms due to length restrictions. For example a train with 13 carriages may only stop at a platform with a length greater or equal than 200 meters. In contrast a train that is only 40 meters long can perhaps stop successfully at all station's platforms. Further, due to the dead end of the tracks, rather strict speed restrictions apply [3]. If the terminus is embedded in some larger environment, a train is limited to maybe 40 km/h close to the terminus, while further away from the terminus there may be no speed restriction after all. Also big termini tend to have some tracks serving them that are using bridges or tunnels. E.g. a train that stops at the easternmost platform of a termini and leaves towards the west should at best not use all switches and blocking all ground tracks. If available, it is rather handy to use a bridge from east to west.

To sum it up termini are very sophisticated *constraint satisfaction problems* (CSP) and rather hard to model in Flatland. Even though some of the mentioned points can definitely be neglected for a start, their mentioning may be helpful for future groups. I decided not to pursue this endeavor and focus on adding mandatory intermediate stations to Flatland. Because if you want a train in Flatland to go from a given start to a goal via a terminus you need a concept of intermediate stations anyway.

2 Environment Representation

Plain Flatland environments that use no extension come with a minimum of facts. That means for each train there is an ASP fact denoting the trains existence as well as a start and goal fact. These two facts link the trains with the positions, where they spawn or supposedly vanish; time restrictions for spawning or despawning may be given as well. In addition the map itself is given by some *cell* facts, which hold the information about the track type within the respective cells of the map.

very basic environment facts

- `train(ID).`
  % denotes the existence of a train
- `start(ID, (Y,X), Departure, Direction).`
  % specifies a train's spawn and earliest possible departure ('*Departure*') and the direction of travel ('*Direction*')
- `end(ID, (Y,X), Arrival).`
  % specifies a train's goal and before which time step ('*Arrival*') to arrive at best
- `cell((Y,X), Track_type).`
  % specifies a cell's track type, e.g. curve, switch, ...

Note that the vertical coordinate is denoted first in Flatland.

To successfully include intermediate stations at first it is necessary to denote where a station is. There are certainly multiple ways to do that in ASP; I came up with the fact *station*, which contains four terms. For one station there are as much station facts needed as there are station's platforms, i.e. a station with five platforms has five station facts. The first term of the station fact is a symbolic constant that holds the station's name. The second term holds the platform's coordinates and the third term the platform number. The fourth term states how many platforms the station has in total. An example can be seen in figure 1.

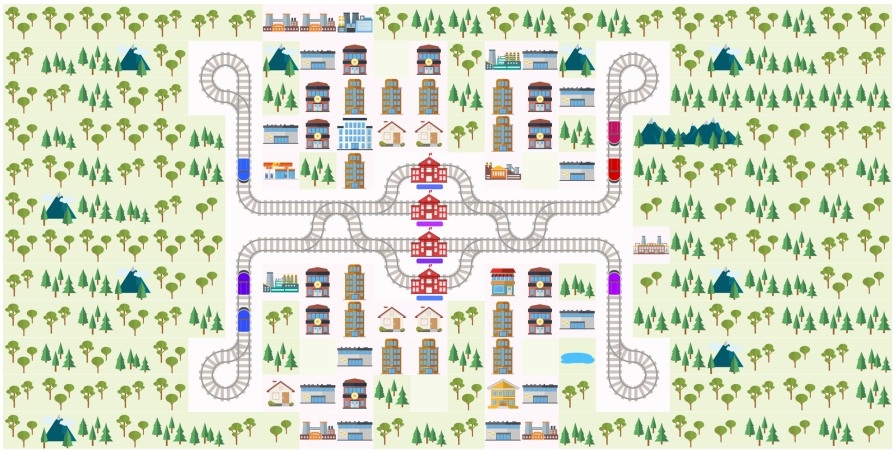


Figure 1: intermediate station *central*
This small map contains my first build intermediate station in the center, symbolized here by the station building icon, that usually represents a train's goal. The trains depart in the west, stop at the station named *central* and arrive at the east or vice versa. The ASP facts are: `station(central, (4,11), 0, 4). station(central, (5,11), 1, 4). station(central, (6,11), 2, 4). station(central, (7,11), 3, 4).`

Once the designer or editor of a Flatland map has added all desired intermediate stations as ASP facts to the environment, the question remains, how trains are using those stations? Therefore I used another fact that is called *stopover*, a term that might be used in an aviation context more frequently. Nonetheless the fact serves the same purpose: If a stopover fact is declared for a train and station, the train has to be routed via that respective station and make a stop there. The stopover fact consists of three entries in total: First the train identifier is given, secondly the station name. And last the mandatory train's dwell time is given, i.e. some integer specifies how many consecutive time steps a train has to stop at the intermediate station before being ready to depart. If in figure 1 train number three should stop at the intermediate station *central* for at least five time steps, the following fact would be provided: `stopover(3, central, 5).`

new environment facts

- `station(StationID, (Y,X), Platform, Total_Platforms).`
  % the station '*StationID*' has a platform at the coordinates (Y,X), this is platform number '*Platform*' overall the station has '*Total_Platforms*' platforms

- `stopover(TrainID, StationID, Dwell_Time).`
  % a train '*TrainID*' has to be routed via station '*StationID*' and stop there for at least '*Dwell_Time*' time steps

If there are multiple intermediate stations at which a train is supposed to stop, there is no possibility yet to enforce a predetermined routing. If for instance train `42` has to stop at the stations *west* and *east* due to some given stopover facts, both stopping first at station *east* and then *west* or vice versa are valid solutions.

3 Encoding

3.1 my personal encoding

At the beginning of the semester all groups were developing an ASP encoding to solve default Flatland instances. My encoding to solve intermediate stations builds up on this very first encoding, since intermediate stations are only an additional feature, which extends plain Flatland.

For my approach I made use of some `position` fact, which holds the position for each train for each time step, unless the train has reached its goal and despawns. Note that in my first encoding all trains were spawning with the first time step, despite this not being mandatory at all, I stuck with this idea. This concept can obviously get problematic if trains share the same spawn coordinates, so I used only environment where this is not the case. After all trains are set to their respective spawns at the very first time step a rather large *choice rule* comes into play. The ASP system *Clingo* can then choose for every train and time step wether to pause, move ahead or if possible to use a switch and thus turn right or left.

ASP Code 1 (choice rule – where to move and if to move at all)**.**
Multiple variations of this choice rule exist; names are shortened here:

```
{
  % move options
  pos(TID, (Y-1, X), Time+1, n_wards): option(Track, Facing, n);
  pos(TID, (Y, X+1), Time+1, e_wards): option(Track, Facing, e);
  pos(TID, (Y+1, X), Time+1, s_wards): option(Track, Facing, s);
  pos(TID, (Y, X-1), Time+1, w_wards): option(Track, Facing, w);


  % wait is only possible at a switch, the start or a station
  pos(TID, (Y,X), Time+1, Facing): switch(Track, Facing, true);
  pos(TID, (Y,X), Time+1, Facing): start(TID,(Y,X), _, _);
  pos(TID, (Y,X), Time+1, Facing): station(_, (Y,X), _, _)
} = 1
  :- pos(TID, (Y,X), Time, Facing), cell((Y,X), Track),
  Time <= maxtime, not is_done(TrainID, Time).
```

The above choice rule (ASP Code 1) is one possible realization to decide, where a train moves or if a train moves after all. The body of the choice rule demands that for every train and time step a new position for the next time step must be found. Unless the train has reached its goal (‘`is_done`’) or the predefined maximum time of the map has been reached (‘`maxtime`’).

The first four lines of the head of the rule enable Clingo to choose a move in any cardinal direction iff the underlying track type allows that movement (cf. `option` fact). In addition Clingo may decide that a train has to wait, but this is only a valid choice if the train is either at a switch, its start or at a station. It should be highlighted that exactly one of the mentioned movement possibilities must be chosen.

One variation of this choice rule is to limit the possibility to choose a *wait move* even further, for instance only allowing it in a station. Or the other way around allowing *waits* in all situations. Another approach is to denote the fact `wait(TrainID, Time)` each time a train has to wait and then letting Clingo minimize the occurrences of these facts. Perhaps even better is to force a train to automatically stop at a station if the train and station are linked through some *stopover* fact. The train then would wait as long as

the *dwell_time* argument within the stopover fact demands. Sadly all these slightly different approaches remained rather slow.

Other than that my different encodings contain some integrity constraints that tackle *vertex* or *edge conflicts*. Also the integrity constrain `:- train(TrainID), not is_done(TrainID, _).` ensures that it can not be the case that a train is not reaching its goal. Additionally the fact *successfully_stopped* is derived if a train stops long enough at an intermediate station to fulfill a stopover fact. The constraint (ASP Code 2) ensures that all stopover fact are fulfilled.

ASP Code 2 (successfully stopped)**.**
it can not be the case that a train is linked to a station via a *stopover* fact and the train has not successfully stopped at this very station:

```
:-  stopover(TrainID, StationID, _),
not successfully_stopped(TrainID, StationID).
```

Finally the general accepted output for a Flatland instance consists of *action* facts. For this purpose the file `convert_to_out_format.lp` derives the desired action facts based on the position facts that my encoding uses. The action facts can then be used to make an animation of the solution with the help of Clingonia [4].

3.2 others work

As my above described approach only slowly finds a solution, the idea came up to use some efficient solution for plain Flatland by somebody else and on top of that implementing intermediate stations. If the third party encoding already uses some sort of *position* fact, one can easily derive the *successfully_stopped* fact as well. Then the constraint given in (ASP Code 2) can directly be implemented within the third party encoding.

On the other hand quite some approaches do not use any kind of fact that would be similar to the *position* fact. Therefore I wrote a file that transforms the standard output of *action* facts of the form `action(train(ID), Movement, Time)` into *was_at* facts, which are similar to my original position facts. Another file then holds a rule to derive the *successfully_stopped* fact and the constraint that it can not be the case that there is a *stopover* fact but no matching *successfully_stopped* fact. The following color box illustrates that concept.

transform others work

1. `file_of_another.lp`
   some third party file that solves default Flatland

2. `convert_to_was_at.lp`
   converts *action* facts to *was_at* facts

3. `supplemental_constrain.lp`
   contains a rule to derive the *successfully_stopped* fact
   contains the following constraint:

```
:- stopover(TrainID, StationID, _),
not successfully_stopped(TrainID, StationID).
```

Despite the fact that all these three files would be called at once, first the third party file has to find a solution. Then the conversion to *was_at* can take place and finally it can be checked if the supplemental constrain is violated. This is why this concept might be rather slow as well.

Using other work can be tricky though, especially if one does not know in advance if the third party encoding is efficient, uses the standard output or is even without errors. Neither with my encoding nor with the help of third party encoding I encountered any satisfying efficient solutions so far, but the amount of third party encoding that I used was rather limited. Nonetheless I except that the concept of converting another person's solution itself is rather slow.

4 Maps

During the semester I built multiple maps that contain intermediate stations. Somehow I liked the idea, that all trains are visible from the very beginning. Therefore in my encoding all trains spawn at the very first time step and I need some waiting space in my maps. Hence I built *waiting loops* in all of the maps, so the map itself limits the order in which trains can be dispatched – if all trains spawn at once. Originally I was thinking of a dispatcher, who is only responsible for a certain geographical area. Presumably the dispatcher has

to deal with a predetermined order (i.e. *queue*) in which trains are entering his area of responsibility.

My favorite map is the *three corridors* map that can be seen in figure 2. Trains either spawn in the north moving southwards or they spawn in the south moving northwards. The map contains two large intermediate stations, one in the north and one in the south, at which all trains have to stop. Further there are the three corridors that lead to the map name. Each train is either linked to one of the three corridors through some stopover facts or assigned to none of the corridors. With a file written in python, one can create random instances from one up to 70 trains. The probability that a train is assigned to one of the three corridors or assigned to none can be specified in the python file. For testing purposes there is also a version of this map that contains only the central corridor.

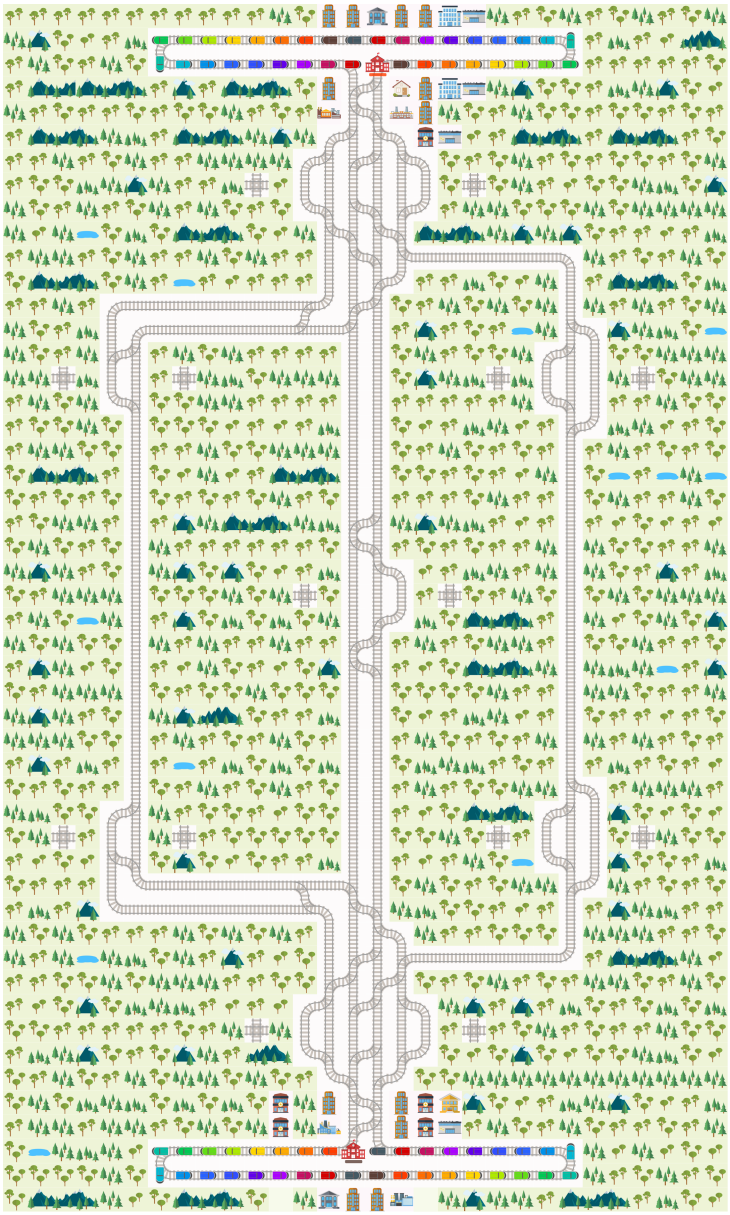


Figure 2: three corridors map
The trains spawn either in the north, heading towards the south or start and goal are reversed. There are to large (intermediate) station, one in the south and one in the north, further each corridor contains some stations. Those stations are all marked by *cross tracks* right next to them. Most trains are assigned to one corridor and its stations throughout the *stopover* facts.

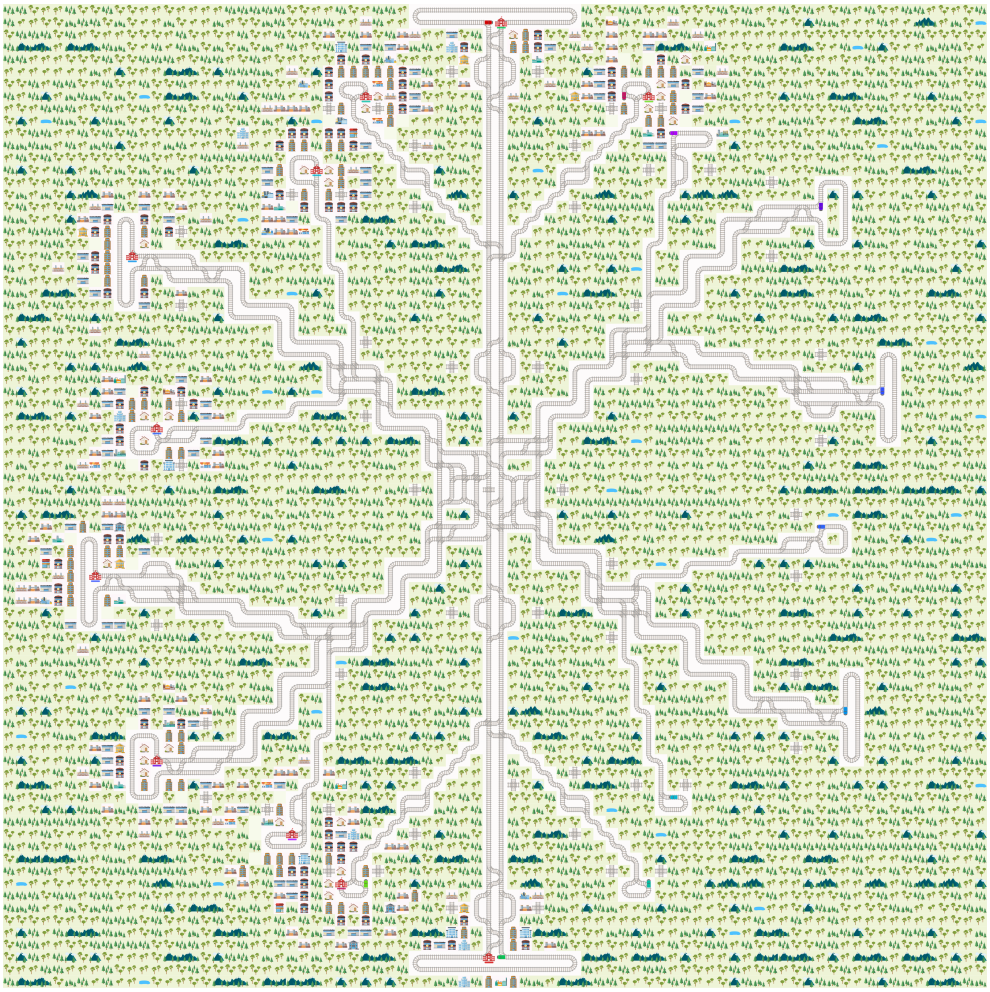


Figure 3: snowflake map

After looking out of the window during some cold January evening I knew what to build. So far this map hasn't been included in testing, you still need to add *stopover* facts and if you so choose, more trains.

The *Clingonia* framework proved really helpful for creating all these maps [4]. Nevertheless the *station* facts, that point out the location of intermediate stations, were added by hand to the environment file.

5 Summary

Presumably the first steps in adding intermediate stations are made. However my encoding for finding a solution for a Flatland instance with intermediate stations is rather slow - often Clingo doesn't find a solution after all. Therefore efficiency might be the most urgent problem regarding intermediate stations in Flatland. Hopefully my facts to denoted intermediate stations, the concept of my encoding as well as the maps I built can be helpful for potential future groups pursuing this topic. Further one could try to extend the *stopover* fact so that trains are forced to stop at stations in some predetermined order. Because currently, it is not possible to force a train to visit a station multiple times, like that would be the case with some *circle line*. Further one could try to combine sophisticated Flatland features like *malfunctions* or *train speed* with intermediate stations.

References

[1] The Flatland Community. website of the Flatland project, url: `https://flatland.aicrowd.com/intro.html`, last access: 17.03.2026.

[2] S. P. Mohanty, E. Nygren, F. Laurent, M. Schneider, C. Scheller, N. Bhattacharya, J. D. Watson, A. Egli, C. Eichenberger, C. Baumberger, G. Vienken, I. Sturm, G. Sartoretti, and G. Spigler. Flatland-rl : multi-agent reinforcement learning on trains. *CoRR*, abs/2012.05893, 2020. DOI: `https://doi.org/10.48550/arXiv.2012.05893`. URL: `https://arxiv.org/abs/2012.05893`.

[3] OpenRailwayMap. map of the world's railway infrastructure, url: `https://www.openrailwaymap.org/`, last access: 17.03.2026.

[4] Clingonia. repository of Clingonia, url: `https://github.com/M4X-VOGEL/Clingonia`, last access: 17.03.2026.